Clean model description

July 22, 2026

1 Model description

Simplified Model

Jobs arrive over time and must be routed to one of two back-end servers:

- **Fast server:** faster processing, but incurs an additional routing cost.
- **Standard server:** slower processing, but no routing cost.

The fast server is not always the best choice because it may already have a long queue. Therefore, the routing decision trades off the lower processing time of the fast server against its routing cost and current congestion.

Each arriving job has two characteristics:

- a *job value* $v \in [0, 1]$, which determines the revenue generated by completing the job;
- a hidden binary state indicating whether the job is *simple* or *complex*.

Simple jobs require relatively little processing regardless of the selected server. In contrast, complex jobs require substantially more processing time, especially when routed to the standard server. Consequently, complex jobs benefit the most from being assigned to the fast server.

The decision maker does not observe whether a job is complex. However, by probing the job, an imperfect signal can be obtained. This signal is informative, but not perfectly accurate, and is used to improve the routing decision.

We consider a family of threshold policies of the form

$$\text{score} = a_v\, v - a_q\, (q_f - q_s) + a_s\, s,$$

where q_f and q_s denote the current workloads (or queue lengths) of the fast and standard servers, respectively, and $s \in \{0, 1\}$ is the probing signal. The job is routed to the fast server whenever

$$\text{score} \geq \tau,$$

where τ is the policy threshold.

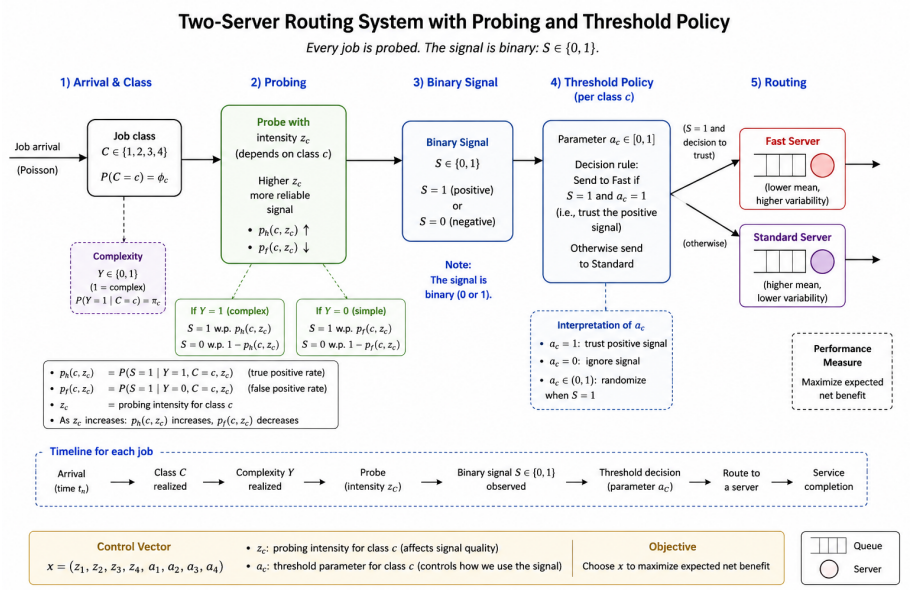


Figure 1: A summary of what is going on!

Example. Suppose

$$a_v = 0.7, \qquad a_q = 0.4, \qquad a_s = 0.8, \qquad \tau = 0.3.$$

Consider a job with value $v = 1$, signal $s = 1$, and equal queue lengths ($q_f = q_s$). The score is

$$0.7(1) - 0.4(0) + 0.8(1) = 1.5 > 0.3,$$

so the policy routes the job to the fast server. On the other hand, if the fast queue becomes substantially longer than the standard queue, the score decreases and the same policy may instead route the job to the standard server.

Job classes and complexity. Jobs belong to one of four classes. Each class has a different probability of being intrinsically complex:

$$\mathbf{p}^{\text{complex}} = (0.02,\ 0.12,\ 0.55,\ 0.95).$$

For example, only 2% of class-1 jobs are complex, whereas 95% of class-4 jobs are complex.

Conditional on the true complexity of the job, the probe generates an imperfect binary signal. Its quality is characterized by two vectors:

$$\mathbf{p}^{\text{hit}} = (0.25,\ 0.40,\ 0.75,\ 0.92), \qquad \mathbf{p}^{\text{false}} = (0.05,\ 0.08,\ 0.15,\ 0.20).$$

The first vector gives the probability that the probe correctly emits a positive signal when the job is complex (true positive), whereas the second gives the

probability that the probe incorrectly emits a positive signal when the job is simple (false positive).

Example. Consider a class-1 job. It is complex with probability 0.02. If it is indeed complex, then the probe correctly emits a positive signal with probability 0.25 (and misses it with probability 0.75). On the other hand, if the job is simple, the probe emits a false positive signal with probability 0.05. [de: Is there any concern with how p_1^{hit} and p_2^{hit} are both less than 0.50?] [bp: Yeah, they are maybe too low, meaning not worth paying for those. We can always change that.]

Probing effort. The decision variable $z_c \in [0, 1]$ represents the probing effort allocated to class-c jobs. This effort is fractional because probing is interpreted as an intensity: a larger value of z_c means that class-c jobs are inspected more carefully, using more time, resources, or diagnostic checks.

Probing improves the quality of the signal. For a class-c job, the true-positive and false-positive probabilities become

$$p_c^{\text{hit}}(z_c) = p_c^{\text{hit}} + (1 - p_c^{\text{hit}}) \left(1 - e^{-\beta z_c}\right),$$

and

$$p_c^{\text{false}}(z_c) = p_c^{\text{false}} e^{-\beta z_c}.$$

Thus, more probing increases the probability of detecting a complex job and decreases the probability of falsely flagging a simple job.

Example. For class 1, suppose

$$p_1^{\text{hit}} = 0.25, \qquad p_1^{\text{false}} = 0.05, \qquad \beta = 3, \qquad z_1 = 0.5.$$

Then

$$p_1^{\text{hit}}(0.5) = 0.25 + 0.75(1 - e^{-1.5}) \approx 0.833,$$

whereas

$$p_1^{\text{false}}(0.5) = 0.05 e^{-1.5} \approx 0.011.$$

Therefore, moderate probing substantially increases the probability of detecting a complex class-1 job, while also reducing false positives.

2 What could we improve/add to the model?

1. Maybe the routing policy should observe job class
2. We should improve the story behind the classes. Something like
   (a) Class 1: routine requests,
   (b) Class 2: moderately difficult,

(c) Class 3: specialized,

(d) Class 4: highly customized.

3. The role of job value is still a bit weak. Right now, it only changes the revenue. It does not affect processing, due dates, or penalties. Consequently, it only influences routing indirectly. We might have a model in which higher-value jobs should also have tighter service requirements or higher tardiness penalties

3 Simulation Procedure

For a fixed probing decision z, the objective function can be interpreted as the following simulation procedure.

1. Generate one realization of the underlying system, consisting of a complete sample path
$$\xi_\ell = (\xi_{\ell 1}, \ldots, \xi_{\ell R}),$$
where each $\xi_{\ell r}$, $r = 1, 2, \ldots, R$, represents one simulated job in the queue and ℓ represents the replication index. This ℓth sample path is generated independently of the probing decision. [de: I took this last point to mean that we initially generate ξ_ℓ unconditionally (i.e., assuming no probing), but please correct me.]

2. Given the probing decision z, generate one realization of the probing outcomes, denoted by η_z. In the present model, η_z contains the probing results for all jobs and is a length-R vector.

3. Repeat the above operation n times, for a total of n independent replications of the simulation model.

4. [de: I paused revising this text/notation here because of my attempts to unify the setups (see below).] For each routing system $i = 1, \ldots, k$, estimate its conditional expected performance given the observed probing outcomes by
$$\frac{1}{n}\sum_{\ell=1}^{n} Y(x_i, \xi_\ell)\frac{f(\xi_\ell; \eta_z)}{f(\xi_\ell)}.$$
The likelihood ratio transforms the unconditional sample paths into an estimator of the conditional expectation given η_z.

5. Select the routing system with the largest conditional expected value,
$$\max_{i=1,\ldots,k} \frac{1}{n}\sum_{\ell=1}^{n} Y(x_i, \xi_\ell)\frac{f(\xi_\ell; \eta_z)}{f(\xi_\ell)},$$
and record this quantity as one realization of the outer expectation.

6. Repeat the previous steps for independent realizations of η_z and average the resulting values to estimate the objective function.

The likelihood-ratio approach exploits the fact that the underlying randomness of the system (arrival process, true job characteristics, service times, etc.) is independent of the probing decision z. Changing z only modifies the distribution of the probing outcomes and, consequently, the information available to the routing systems. As a result, the routing decisions and the realized system trajectory change with z, even though they are driven by the same underlying realization. This separation allows the underlying realizations to be recycled while accounting for different probing decisions through likelihood-ratio weights.

Saying the same thing again:

1. Nature generates the 200-job day.
2. z changes what the controller gets to observe.
3. The routing systems react differently because they receive different signals.
4. Therefore, the realized queues differ, despite sharing the same underlying randomness.

Could we have done things differently? Yeah, probably. Maybe probing would be a marketing effort that would increase the average number of jobs and even their value.

4 Is that what is in the mysterious Dr. Eckman's mind?

Since the probing decision z does not modify the underlying stochastic process (arrival process, true job characteristics, service times, etc.), the unconditional distribution of the sample paths remains unchanged. The only distribution affected by z is that of the probing outcomes.

Consequently, Bayes' theorem yields

$$f(\xi \mid \eta, z) = \frac{f(\xi)\, f(\eta \mid \xi, z)}{f_z(\eta)},$$

or equivalently,

$$\frac{f(\xi \mid \eta, z)}{f(\xi)} = \frac{f(\eta \mid \xi, z)}{f_z(\eta)}.$$

Substituting this expression into our famous equation(7) gives

$$J(z) = -\alpha^\top z + \int_{\mathbb{R}^{d_z}} \left[\max_{i=1,\dots,k} \frac{1}{n} \sum_{\ell=1}^{n} Y(x_i, \xi_\ell) \frac{f(\eta_z \mid \xi_\ell)}{f_z(\eta_z)} \right] f_z(\eta_z)\, d\eta_z.$$

Since $f_z(\eta_z)$ is independent of the routing system i, it follows that

$$f_z(\eta_z) \max_i \left\{ \frac{A_i(\eta_z)}{f_z(\eta_z)} \right\} = \max_i A_i(\eta_z),$$

and therefore

$$\boxed{J(z) = -\alpha^\top z + \int_{\mathbb{R}^{d_z}} \max_{i=1,\ldots,k} \left[\frac{1}{n} \sum_{\ell=1}^{n} Y(x_i, \xi_\ell) f(\eta_z \mid \xi_\ell) \right] d\eta_z.}$$

I like this derivation because it isolates exactly where the simplification comes from: not from the likelihood ratio itself, but from the fact that $f(\xi)$ is independent of z. The marginal density $f_x(\eta)$ appears naturally through Bayes' rule and then cancels against the outer expectation. This is a much cleaner argument than simply asserting that the likelihood ratio simplifies the expression.

[de: The name of that last section really cracked me up.

I see things a little differently, as I'll try to describe here. It may be that things end up at the same place as the analysis above, but by taking a different path.]

[bp: Believe me, Franco and I asked ourselves a few times "what does he mean by simplifying the expression???"]

Let ξ represent the aleatoric uncertainty in a single replication. In our example, ξ consists of the arrival times, service times (both in the first stage and the two possibilities in the second stage—at the fast and at the standard server, even though only one of these values will be used), the classes of the jobs, the complexities of the jobs, and the signals (probing outcomes) for all jobs. IMPORTANT: The signal MUST be part of ξ because the routing decision (the simulation model's logic) depends on the signal.

In terms of our notation, η_z is the list of probing outcomes/signals (one for each job) *given a probing decision* z. Hence, one component of ξ equals η_z. [de: I believe you mentioned one of the first papers on probing making such an assumption. The fact that we have this degeneracy in our problem somewhat distorts the derivations of the likelihood ratios and gradient estimators; see what is to come. I think that is find for an initial example, but I believe the setup we envisage is the more general one, where η does not directly enter into the simulation model's logic.] [bp: I see, I see. So. you mean that the general case would be something like

$$\xi = (\lambda_z, \mu_z, \theta_z, \nu_z, \eta_z),$$

right???] [de: Yes. There may be some advantage in defining $\xi = (\xi_{-z}, \xi_z)$ where the portion ξ_z depends on z and the portion ξ_{-z} does not. In our example, $\xi_{-z} = (\lambda, \mu, \theta, \nu)$ while $\xi_z = \eta_z$.]

[bp: How would things change if we had, say, $\xi_z = (\lambda_z, \eta_z)$? Just a harder gradient computation? Maybe having to use simulation for it?]

[bp: It seems the first big change (what we called level II in our discussions) would be z generating a distribution on the values of ξ_z, which would require the famous external integral of the even most famous equation (7). It seems messier, but essentially the same logic.]

[bp: The different setting would really be z changing the system. In summary, I see things like that now:

Distributional probing:	$z \longrightarrow \xi_z$	$\xi_z \sim P(\cdot; z)$,
Random parameter	$z \longrightarrow \theta_z \longrightarrow \xi$	$\theta_z \sim P_\theta(\cdot; z), \quad \xi \sim P_\xi(\cdot \mid \theta_z)$,
Configuration/design:	$z \longrightarrow \mathcal{C}(z)$	$Y = Y(x, \xi; \mathcal{C}(z))$.

]

[de: Note: For the next part, we will be talking about a single replication and will suppress the replication index ℓ from the subscript on ξ. It will come back later when we write the overall LR estimator.] Assume, for simplicity, that the stochastic elements are independent across jobs, i.e., $\xi_1, \xi_2, \ldots, \xi_R$ are independent. Within a given ξ_r (job r), however, the stochastic elements are not independent. For example, the complexity of the job depends on the problem class, and the probing outcome (signal) depends on the complexity and the job class. The service times also depend on the complexities and the job classes, but this is less important.

Let $f(\xi)$ represent the *unconditional* density of the stochastic primitives, assuming NO PROBING. The probing outcomes therefore follow their default distributions when $z = \mathbf{0}$. As a result of the assumption of independence across jobs, the density $f(\xi)$ decomposes into a product of densities: $f(\xi_1) \times f(\xi_2) \times \cdots \times f(\xi_R)$. In what follows, we focus on one such $f(\xi_r)$.

$$
\begin{aligned}
f(\xi_r) &= f(\text{arrival, services, complexity, class, signal}) \\
&= f(\text{arrival, services} \mid \text{complexity, class}) \times f(\text{signal} \mid \text{complexity, class}) \\
&\quad \times f(\text{complexity} \mid \text{class}) \times f(\text{class})
\end{aligned}
$$

Of the four terms in $f(\xi_r)$, signal appears only in the second, $f(\text{signal} \mid \text{complexity, class})$. By defining signal $= \eta_{z,r}$, we also have that

$$
\begin{aligned}
f(\xi_r; \eta_{z,r}) = f(&\text{arrivals, services} \mid \text{complexity, class}) \times 1 \\
&\times f(\text{complexity} \mid \text{class}) \times f(\text{class}).
\end{aligned}
$$

The (incomplete, as you will see) likelihood ratio is then

$$
\frac{f(\xi; \eta_z)}{f(\xi)} = \prod_{r=1}^{R} \frac{f(\xi_i; \eta_{z,r})}{f(\xi_r)} = \prod_{r=1}^{R} \frac{1}{f(\text{signal} \mid \text{complexity, class}, z = \mathbf{0})}.
$$

We further need to incorporate $f(\eta_{z,r}; z)$ into the numerator of the likelihood ratio. This too will be *conditional on the complexity and class*, which is not

clearly represented in the notation at this point. [de: My current thinking is that since $\eta_{z,r}$ appears in ξ_r, the likelihood ratio we really want should condition on z rather than η_z (see the upcoming analysis).] What this will lead to is a likelihood ratio of the form

$$\frac{f(\xi; z)}{f(\xi)} = \prod_{r=1}^{R} \frac{f(\text{signal} \mid \text{complexity, class}, z)}{f(\text{signal} \mid \text{complexity, class}, z = \mathbf{0})}.$$

Equation (7) should then look like

$$\boxed{J(z) = -\alpha^\top z + \max_{i=1,\ldots,k} \left[\frac{1}{n} \sum_{\ell=1}^{n} Y(x_i, \xi_\ell) \frac{f(\xi_\ell; z)}{f(\xi_\ell)} \right].}$$

[de: Some new notation.] For job $r = 1, 2, \ldots, R$, let C_r be its class, D_r be its complexity, S_r be its signal. Here, the upper-case letters will denote the *random* interpretations of these quantities and the lower-case letters (c_r, d_r, s_r), will denote the realized values. The realized values will be needed later in the likelihood ratios.

The LR in this problem is

$$\begin{aligned}
\frac{f(\xi_\ell; z)}{f(\xi_\ell)} &= \prod_{r=1}^{R} \frac{P(S_r = s_r | D_r = d_r, C_r = c_r; z)}{P(S_r = s_r | D_r = d_r, C_r = c_r; \text{no probing})} \\
&= \prod_{r:\ \substack{s_r=\text{positive} \\ d_r=\text{complex}}}^{R} \frac{p^{\text{hit}}_{c_r}(z_{c_r})}{p^{\text{hit}}_{c_r}(0)} \prod_{r:\ \substack{s_r=\text{negative} \\ d_r=\text{complex}}}^{R} \frac{1 - p^{\text{hit}}_{c_r}(z_{c_r})}{1 - p^{\text{hit}}_{c_r}(0)} \\
&\quad \times \prod_{r:\ \substack{s_r=\text{positive} \\ d_r=\text{simple}}}^{R} \frac{p^{\text{false}}_{c_r}(z_{c_r})}{p^{\text{false}}_{c_r}(0)} \prod_{r:\ \substack{s_r=\text{negative} \\ d_r=\text{simple}}}^{R} \frac{1 - p^{\text{false}}_{c_r}(z_{c_r})}{1 - p^{\text{false}}_{c_r}(0)}
\end{aligned}$$

[de: REGARDE ICI: The blue LR was previously missing from the expression for $\nabla_z J(z)$ below. I am fairly certain it needs to be there, because otherwise the red term alone has no dependence on the z under which the ξ's were simulated, which does not make sense.] The associated LR gradient estimator is

$$\boxed{\nabla_z J(z) = -\alpha + \frac{1}{n} \sum_{\ell=1}^{n} Y(x_{i^*}, \xi_\ell) \frac{f(\xi_\ell; z)}{f(\xi_\ell)} \nabla_z \log f(\xi_\ell; z)}$$

where

$$i^* = \arg \max_{i=1,\ldots,k} \left[\frac{1}{n} \sum_{\ell=1}^{n} Y(x_i, \xi_\ell) \frac{f(\xi_\ell; z)}{f(\xi_\ell)} \right].$$

The score function term in this problem is

$$\begin{aligned}
\nabla_z \log f(\xi_\ell; z) &= \sum_{r=1}^{R} \nabla_z \log P(S_r = s_r | D_r = d_r, C_r = c_r; z) \\
&= \sum_{\substack{r:\ s_r=\text{positive} \\ d_r=\text{complex}}}^{R} \nabla_z \log p_{c_r}^{\text{hit}}(z_{c_r}) + \sum_{\substack{r:\ s_r=\text{negative} \\ d_r=\text{complex}}}^{R} \nabla_z \log \left(1 - p_{c_r}^{\text{hit}}(z_{c_r})\right) \\
&\quad + \sum_{\substack{r:\ s_r=\text{positive} \\ d_r=\text{simple}}}^{R} \nabla_z \log p_{c_r}^{\text{false}}(z_{c_r}) + \sum_{\substack{r:\ s_r=\text{negative} \\ d_r=\text{simple}}}^{R} \nabla_z \log \left(1 - p_{c_r}^{\text{false}}(z_{c_r})\right) \\
&= \sum_{\substack{r:\ s_r=\text{positive} \\ d_r=\text{complex}}}^{R} \frac{(1 - p_{c_r}^{\text{hit}}(0))\beta e^{-\beta z_{c_r}}}{p_{c_r}^{\text{hit}}(z_{c_r})} + \sum_{\substack{r:\ s_r=\text{negative} \\ d_r=\text{complex}}}^{R} \frac{-(1 - p_{c_r}^{\text{hit}}(0))\beta e^{-\beta z_{c_r}}}{1 - p_{c_r}^{\text{hit}}(z_{c_r})} \\
&\quad + \sum_{\substack{r:\ s_r=\text{positive} \\ d_r=\text{simple}}}^{R} \frac{-p_{c_r}^{\text{false}}(0)\beta e^{-\beta z_{c_r}}}{p_{c_r}^{\text{false}}(z_{c_r})} + \sum_{\substack{r:\ s_r=\text{negative} \\ d_r=\text{simple}}}^{R} \frac{p_{c_r}^{\text{false}}(0)\beta e^{-\beta z_{c_r}}}{1 - p_{c_r}^{\text{false}}(z_{c_r})}
\end{aligned}$$

[de: New text about the green gradient search.]

The LR gradient estimator $\nabla_z J(z)$ derived above can be evaluated at any z. Hence, we can think of $J(z)$ as a surrogate for the objective function, which we can then attempt to optimize using a gradient based search. In the setup we have tested, we perform n initial replications under the no-probing case and obtain a static surrogate J that we then optimize. We know J will be biased and (possibly) highly variable for some z, like those z far from 0. As we increase n, we expect the bias and variable to both vanish, but possibly at a slow rate for z far from 0.

We could alternatively take n observations at each iterate z^τ ($\tau = 0, 1, 2, \dots$) visited by the search and use only these n observations to compute a LR gradient estimate and thereby take a gradient step. The advantage of this approach is that LR gradient estimator should be more accurate because it is based on replications under the current z (hence the likelihood ratio in $\nabla_z J(z)$ is uniformly 1 and therefore vanishes). The downside of this approach is that it essentially discards all previous simulation replications taken under past probing decisions.

The idea behind green simulation is to achieve a greater effective sample size (and therefore a less variable LR gradient estimator) by reuse historical observations previously collected throughout the search. These observations are appropriately reweighted by a LR and score function, both of which depend on the current iterate: the z^τ where the gradient is being estimated. [de: Aside: An early paper of mine on the green gradient algorithm demonstrated that there is an additional bias introduced by this reuse due to the observations from different iterates being *dependent*. This dependence arises because the search trajectory (and hence the observations you will see next) depends on the observations you already saw.]

Suppose we have run a gradient search in which we have visited a sequence of probing decisions $z^0, z^1, \ldots, z^t$ and have generated n i.i.d. observations at each of these iterates. The green LR gradient estimator for the gradient at z^t is

$$\boxed{\nabla_z J^{\text{green}}(z^t) = -\alpha + \frac{1}{t+1}\sum_{\tau=0}^{t}\left[\frac{1}{n}\sum_{\ell=1}^{n} Y(x_{i^*}, \xi_\ell; z^\tau)\frac{f(\xi_\ell; z^t)}{f(\xi_\ell; z^\tau)}\nabla_z \log f(\xi_\ell; z^t)\right]}$$

where the z^τ in the notation $Y(x_{i^*}, \xi_\ell; z^\tau)$ reflects that this observation was obtained when ξ_ℓ was generated under the conditional distribution associated with z_τ, i.e., it's one of the n outputs obtained when running the simulation model under a probing decision z^τ previously visited during the gradient search. The score function $\nabla_z \log f(\xi_\ell; z)$ is the same as that shown above, and the likelihood ratio $\frac{f(\xi_\ell; z^t)}{f(\xi_\ell; z^\tau)}$ is nearly the same as that shown above, except with $p_{c_r}^{\text{hit}}(z_{c_r}^\tau)$ and $p_{c_r}^{\text{false}}(z_{c_r}^\tau)$ in the denominators instead of $p_{c_r}^{\text{hit}}(0)$ and $p_{c_r}^{\text{false}}(0)$.

5 Notes on Differences Compared to Equation (7)

[de: This text is new.] When we first derived Equation (7), the original problem was

$$\max_{z\in[0,1]^m}\left\{-\alpha^T z + \mathbb{E}_\eta\left[\max_{i=1,2,\ldots,k}\mathbb{E}\left[Y(x_i,\xi)\mid \eta; z\right]\right]\right\}.$$

In this setup, we were able to first observe η (from probing) and then decide on an optimal ACTION i before experiencing a reward Y that depended on further randomness ξ. In our production example, on the other hand, we have to commit to a POLICY beforehand when we choose the coefficients and thresholds. This policy then dictates which action we take when we see a certain probing signal η. In the production example, i corresponds to the policy parameters, not the decision to route the job to the standard or fast line. The examples we've been looking at recently do not have the full "two-stage" setup where we pick z, observe η, pick i, and observe ξ. Instead, we are simultaneously picking z and i and then experiencing ξ (which now includes η). Put differently, there is no expectation outside the maximum, hence our formulas for the LR estimators did not involve the Monte Carlo integration that is present in the more general two-stage model we started with.

6 Example 2: Predictive Maintenance with Continuous Inspection Effort

[de: My two cents: I like the story told by this example. Other than the use of normal distributions instead of Bernoulli's in the likelihood ratios, I'm not sure it's fundamentally different from the production example. Still, I really like it.]

[bp: Well, I did not have time to explain why I came up with this example. I FULLY agree it is quite similar to the other example. My goal was to dig deeper into the story of what we are doing. Before coming with this example I tried a lot of other ideas that did not work out. I tried to adapt some well-known problems in the POMDP literature in which you have information acquisition/learning, but they felt flat. The trade-off was just not there, or at least I could not see it. This one seems to be a better blueprint for what we want to do.]

Consider a system consisting of machines that are subject to degradation and unexpected failures. At an inspection epoch, each machine has an observable age $A \in [0,1]$ and an unobservable degradation level $D \in [0,1]$. Older machines tend to be more degraded, but age alone does not perfectly reveal the true condition of a machine. We model degradation as

$$D = [0.10 + 0.72A + 0.16Z_D]_{[0,1]}, \qquad Z_D \sim N(0,1),$$

where $[\cdot]_{[0,1]}$ denotes truncation to the interval $[0,1]$.

The decision maker can acquire additional information about the condition of the machine through an inspection. Inspection produces a noisy signal

$$S = D + \varepsilon,$$

where the precision of the signal depends on the inspection effort.

We consider two age groups,

$$g(A) = \begin{cases} 1, & A < 0.55, \\ 2, & A \geq 0.55, \end{cases}$$

corresponding to younger and older machines, respectively. The probing decision is

$$z = (z_1, z_2) \in [0,1]^2,$$

where z_1 is the inspection effort devoted to younger machines and z_2 is the inspection effort devoted to older machines.

Conditional on the true degradation level D, the inspection signal satisfies

$$S \mid D, z_g \sim N\left(D, \sigma_g^2(z_g)\right),$$

where

$$\sigma_g(z_g) = \sigma_{0,g} \exp(-\beta_g z_g).$$

for $g = 1, 2$. Thus, increasing z_g reduces the variance of the inspection error and provides more accurate information about the true condition of the machine. Inspection is costly, with total information-acquisition cost

$$C(z) = \alpha_1 z_1 + \alpha_2 z_2.$$

After observing the machine's age A and inspection signal S, the decision maker chooses whether to perform preventive maintenance or to keep the machine in operation. We consider a family of threshold policies of the form

$$\pi_\theta(A, S) = \begin{cases} \text{preventive maintenance}, & a_S S + a_A A \geq \tau, \\ \text{continue operation}, & a_S S + a_A A < \tau, \end{cases}$$

where

$$\theta = (a_S, a_A, \tau)$$

denotes the parameters of the maintenance policy.

The threshold policy combines two sources of information. Age A is observed freely and provides prior information about machine condition, whereas S is costly information whose accuracy depends on the probing decision z.

If preventive maintenance is performed, a moderate and predictable maintenance cost is incurred. In the numerical experiment, this cost is approximately

$$C_{\mathrm{PM}} = 18.$$

Preventive maintenance may therefore be undesirable when performed too early, since a machine with substantial remaining useful life is replaced or repaired unnecessarily.

If preventive maintenance is not performed, the machine continues operating and may experience an unexpected failure. Conditional on age and degradation, the failure hazard is modeled as

$$\lambda(A, D) = \lambda_0 \exp\left(\gamma_D D + \gamma_A A\right).$$

A potential failure time is generated according to

$$T_F \mid A, D \sim \text{Exponential}\big(\lambda(A, D)\big).$$

If T_F occurs within the operating horizon, the machine fails unexpectedly. Such a failure is much more expensive than preventive maintenance, with a corrective-maintenance cost of approximately

$$C_{\mathrm{fail}} = 105,$$

in addition to production losses associated with unplanned downtime.

The fundamental trade-off is therefore

premature preventive maintenance	versus	costly unexpected failure.

Inspection information is valuable because it helps distinguish machines that should be maintained from machines that can safely remain in operation. However, more accurate information requires greater inspection expenditure. The outer optimization problem therefore determines how much information should be acquired:

$$\max_{z \in [0,1]^2} \left\{ \max_{\theta \in \Theta} \mathbb{E}\left[Y(\theta, \xi) \mid z\right] - \alpha^\top z \right\},$$

where Θ is the finite set of candidate threshold policies and ξ contains the random machine characteristics, inspection signal, and failure time.

[de: For "random machine characteristics," I assume this refers to the machine's age and degradation. Because age is observable, we may want to treat it as deterministic. As for the degradation, to run a replication, we would need

a probability model for the distribution of D, for the sake of generating random machine characteristics. But where would this model come from? It would seem to correspond to the no-probing case ($z = 0$). The inspection signal and failure time both correspond to aleatoric uncertainty (risk), whereas the random degradation is more like our epistemic uncertainty (uncertainty about the state of the world). [In the production example, we knew the underlying probability of jobs of certain classes being simple or complex and probing only told us something about the individual jobs.] In this example, only the inspection signal depends on probing, so again S is the natural choice for the η in our notation. Like in the production example, the simulation code needs the signal, so S is also part of ξ and is fully observed given a probing decision. Long story short, I think this example has the same setup as the production example, where the outer integral goes away.]

[bp: Let's see, let me clarify all that. First, $\xi_z = S, \xi_{-z} = (A, D, T_F)$. As I said before, I fully agree this belong to the same level (I), same spirit, same setup, same category of the factory example. Zero disgreement here. Regarding Age, I think it could be deterministic, or we could generate ages randomly. I see this more as a modelling choice.

Yes, D requires an underlying degradation model. My interpretation is that

$$D \mid A \sim P_D(\cdot \mid A)$$

represents the baseline distribution of the true physical degradation among machines of a given observed age. In a real application, this distribution would be estimated or calibrated from historical reliability, inspection, or condition-monitoring data.

Importantly, this is not specifically a "no-probing" ($z = 0$) distribution. The machine has a realized true degradation state D regardless of whether we inspect it. The probing decision does not generate or modify D; it only determines how accurately we observe its realized value through the inspection signal:

$$D \mid A \sim P_D(\cdot \mid A), \qquad S \mid D, z \sim P_S(\cdot \mid D, z).$$

Thus, D belongs to the baseline uncertainty of the physical system and is independent of z, whereas S is the z-dependent random variable. In our numerical example, we simply posit a synthetic model for $D \mid A$ for simulation purposes. I agree that S therefore plays the same role as the probing signal in the production example, and the example has the same basic structure with respect to the probing decision.

]

[de: Some details of what constitutes a replication are still unclear to me. Is there some finite time horizon and a fixed set of machines and we simultaneously make decisions to repair/not repair each machine and then watch them break down (or not) and compute a cost.]

[bp: Yeah, well, this was mostly a quick placeholder. Details are in the code! Here is a more detailed explanation. If you like it, feel free to add to the body of the text: A replication is simpler than that. In the current implementation, one

replication corresponds to one machine at one inspection epoch. The machine has an observed age A and a latent degradation state D. Given the probing decision z, an inspection signal S is generated, and the threshold policy then decides whether to perform preventive maintenance or to continue operating the machine. If preventive maintenance is performed, the corresponding preventive maintenance cost is incurred and the replication ends. Otherwise, the machine is operated over a fixed finite horizon (normalized to one time unit). A potential failure time T_F is generated from a distribution whose hazard depends on the machine's age and true degradation. If T_F falls within the operating horizon, an unexpected failure occurs and the corresponding failure and downtime costs are incurred; otherwise, the machine survives the horizon.

Thus, one replication can be summarized as

$$(A, D) \longrightarrow S(z) \longrightarrow \{\text{preventive maintenance or continue}\} \longrightarrow \{\text{survive or fail over the operating horizon}\}.$$

The simulation then averages the cost over many independent machine episodes. There is currently no fixed fleet of interacting machines, no maintenance capacity constraint, and no queue for maintenance.]

[de: This may be a crazy idea, but it may be possible to extend the model to a continuous monitoring setting. Please bear with me... Age would increase linearly with time, i.e., $A(t) = A_0 + t$ where A_0 is the age of the machine at the start of the time horizon. The signal would be continuously monitored where $\varepsilon(z_{g(A)})$ is now a drift-zero Brownian motion with diffusion $\sigma_g^2(z_g)$. (A nuisance with this is that the machine's group g could change over time, which changes the diffusion discontinuously, but whatever.) We would also need another model for how the degradation increases over time (with greater use) and how the machine fails; the current failure hazard model assumes the age and degradation of the machine are fixed, whereas we would now have both increase over time. This is all stream-of-conscious, so please just ignore this for now.]

[bp: Ha! I like it! If you read my previous comments (I wrote things in order without verifying what you wrote after) you sort of fulfilled my prophecy of "it makes sense to play around a bit more and propose new examples of the type 1. Nice ideas could come out of that." I will try to have something VERY RUDIMENTARY based on your example for the meeting.]

The sequence of events for a machine can be summarized as

$$\boxed{\begin{aligned}(A, D) \;&\longrightarrow\; z_{g(A)} \;\longrightarrow\; S = D + \varepsilon(z_{g(A)}) \text{ where } \varepsilon(z_{g(A)}) \sim N(0, \sigma_g^2(z_g)) \\ &\longrightarrow\; \pi_\theta(A, S) \;\longrightarrow\; \begin{cases}\text{preventive maintenance,} \\ \text{continued operation and possible failure.}\end{cases}\end{aligned}}$$

The role of continuous probing is particularly transparent in this example. At low inspection effort, the signal S is noisy, so maintenance decisions rely heavily on age and may result either in unnecessary preventive maintenance or missed detection of highly degraded machines. Increasing inspection effort

improves the accuracy of S, allowing a better maintenance policy to be implemented. Beyond some point, however, the marginal value of additional precision may be smaller than its marginal inspection cost, potentially leading to an interior optimal probing decision z^*.

7 Example 3: the well-known Eckman maintenance model

Here are some very preliminary results for the well-known Eckman continuous maintenance problem:

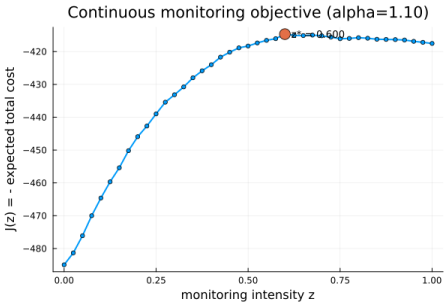


Figure 2: (a) Objective function as a function of monitoring intensity z.

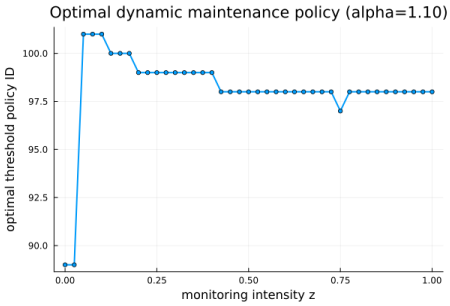


Figure 3: (b) Optimal dynamic maintenance policy as a function of monitoring intensity z.

Figure 4: Continuous-monitoring predictive maintenance. Increasing the monitoring intensity z improves the quality of information about machine degradation but incurs a higher monitoring cost. The resulting trade-off yields an interior optimal monitoring intensity of approximately $z^* = 0.60$.